

02_vision_generative

Local Vision + Generative Media Stack on RTX 5090 (32 GB, Blackwell), Linux, Rootless Podman

Research date: 2026-07-06 **Target hardware:** 1× NVIDIA RTX 5090 (32 GB GDDR7, Blackwell sm_120), Linux host, all workloads in **rootless Podman** containers.
Goal: give a local LLM system (“HelixLLM”) full vision + generative media capability (image understanding, image gen, video gen incl. WAN + LTX, vectorization) behind **OpenAI-compatible** endpoints so Claude Code / HelixCode auto-recognize them.

Evidence discipline (§11.4.6 / §11.4.99): every recommendation below carries a source URL + access date. Numbers taken from third-party benchmarks are marked as such; where a figure could not be independently corroborated it is flagged **UNCONFIRMED**. Blackwell (sm_120) is a 2025 architecture — several toolchains still require nightly/cu128 builds; those caveats are called out explicitly.

0. Executive summary — recommended stack

Capability	Recommended local model	Serving engine (rootless container)	Standard API surface
VLM (image/screenshot/OCR understanding)	Qwen3-VL-8B-Instruct (all-rounder) or Qwen3-VL-30B-A3B (MoE, more capable, still fits)	vLLM (throughput, native OpenAI server) or llama.cpp llama-server + mmproj GGUF (simplicity)	POST /v1/chat/completions with image content parts
Image generation	FLUX.1-dev (fp8) for quality + FLUX.1-schnell for speed; SDXL /	ComfyUI (workflow engine) + OpenAI-image bridge node	POST /v1/images/generations (via bridge) or ComfyUI / prompt

Capability	Recommended local model	Serving engine (rootless container)	Standard API surface
	SD3.5-Large as alternates		
Video generation	LTX-Video / LTX-2 (fast, near-real-time) + WAN 2.2 14B (fp8) (quality)	ComfyUI (WanVideoWrapper / native LTX nodes) or LTX self-host server	LTX self-host OpenAI-compatible base_url; ComfyUI / prompt for WAN
Vectorization / illustration	vtracer (raster → SVG, deterministic) + StarVector-8B (ML im2svg) + potrace (mono)	vtracer as CLI microservice; StarVector via vLLM/transformers (it is a VLM)	Small FastAPI wrapper → POST /vectorize; StarVector via /v1/chat/completions

All four capability servers run as **separate rootless Podman containers** sharing the single RTX 5090 via CDI (`--device nvidia.com/gpu=all`). VRAM is time-shared, not partitioned — run them **serially per request class** or accept the memory-pressure risk (see §7 risks).

1. Vision-Language Models (VLMs)

1.1 What fits in 32 GB and what to pick

Qwen3-VL is the 2026 successor to Qwen2.5-VL and is currently the local VLM to beat. Sizes: 2B, 4B, 8B (dense), 30B-A3B (MoE, ~3B active), 32B (dense), 235B (MoE, does **not** fit). ([QwenLM/Qwen3-VL GitHub](#), accessed 2026-07-06; [Local AI Master — Qwen3-VL local setup](#), accessed 2026-07-06)

Model	Params	Approx. VRAM (quant)	Fits 32 GB?	Notes / benchmark
Qwen3-VL-4B	4B dense	~6 GB (Q4_K_M)	huge headroom	edge/fast
Qwen3-VL-8B	8B dense	~12 GB (Q4_K_M); 12–16 GB for large images		“best local all-rounder”; DocVQA leader at size;

Model	Params	Approx. VRAM (quant)	Fits 32 GB?	Notes / benchmark
			☑	MathVista 68.2 → 85.8 vs 2.5 (codersera , accessed 2026-07-06)
Qwen3-VL-30B-A3B	30B MoE / 3B active	~14–16 GB	☑ ☑	best capability/VRAM ratio; MoE keeps active set small
Qwen3-VL-32B	32B dense	21+ GB (Q4/FP8)	(tight w/ big KV)	max dense quality
Qwen2.5-VL-32B	32B dense	~fits (FP8/Q4)	×	prior gen, strong; (apidog , accessed 2026-07-06)
Qwen2.5-VL-72B	72B dense	~72 GB+ (FP8)		needs multi-GPU

VRAM figures: ([codersera](#), [Unsloth Qwen3-VL run guide](#), [Local AI Master](#); all accessed 2026-07-06).

Other 32 GB-capable VLMs (all runnable via llama.cpp mmpoj GGUF — see §1.2): InternVL 2.5/3 (1B–14B), MiniCPM-V (~8B), Gemma 3 (4B/12B/27B), SmolVLM, Pixtral-12B, Mistral-Small-3.1-24B, Llama-4-Scout-17B, Moondream2. ([llama.cpp multimodal.md](#), accessed 2026-07-06). Llama-3.2-Vision (11B/90B) is supported in the broader ecosystem but Qwen3-VL now outclasses it on doc/chart/OCR — **UNCONFIRMED** whether 90B is worth the multi-GPU cost; not recommended here.

Throughput reference (Qwen2.5-VL-7B, NVIDIA A100, vLLM, concurrency 50): 20.89 req/s for images, 7.35 req/s for video. RTX 5090 has lower memory bandwidth than A100 but native FP8 Tensor Cores — treat A100 numbers as a same-order proxy, not identical. ([vLLM issue #24728 multi-modal benchmark](#), accessed 2026-07-06)

1.2 Serving engines

Option A — vLLM (recommended for throughput + clean OpenAI server).

vLLM ships a native OpenAI-compatible server (`vllm serve <model> → /v1/chat/completions` with image parts) and supports Qwen2.5-VL / Qwen3-VL and SGLang-class multimodal. On Blackwell (sm_120) vLLM needs **PyTorch 2.9.0 + CUDA 12.8 (cu128)** or nightly; stock wheels only cover $\leq$ sm_90. Working community config: `TORCH_CUDA_ARCH_LIST="12.0+PTX", NVCC_GENCODE="-gencode=arch=compute_120,code=sm_120"`. FP8 native on Blackwell (bare-metal Linux; note the WSL2 FP8 fallback caveat). ([vLLM issue #13306 RTX 5090](#); [vLLM issue #37242 sm 120 benchmarks](#); [vLLM forum: torch 2.9.0 cu128 working](#); all accessed 2026-07-06)

Option B — SGLang. Documented Qwen2.5-VL path with OpenAI-compatible endpoints and `sglang.bench_serving` tooling; comparable to vLLM. ([SGLang Qwen2.5-VL cookbook](#), accessed 2026-07-06)

Option C — llama.cpp llama-server + mmpromj GGUF (recommended for simplicity/portability). Vision via a separate multimodal projector (`--mmpromj`). One command exposes an OpenAI-compatible `/v1/chat/completions` that accepts images. Runs the whole Qwen3-VL family from ready-made `ggml-org / Qwen/*`-GGUF repos. ([llama.cpp multimodal.md](#); [Qwen3-VL-8B-Instruct-GGUF](#); [Simon Willison — llama.cpp vision](#); all accessed 2026-07-06)

Recommendation: expose vision through **Qwen3-VL-8B** as the default (fast, 12–16 GB), with **Qwen3-VL-30B-A3B** or **-32B** as a “high-quality” profile loaded on demand. Use **vLLM** if you want max concurrency and native FP8; use **llama.cpp** if you want a single self-contained binary and easy GGUF weight management.

```
# llama.cpp OpenAI-compatible vision server (GGUF + mmpromj)
llama-server -hf Qwen/Qwen3-VL-8B-Instruct-GGUF \
  --mmpromj-url <mmpromj.gguf> \
  --host 0.0.0.0 --port 8090 --ngl 999 --ctx-size 32768
# -> POST http://localhost:8090/v1/chat/completions (image_url
      content parts)

# vLLM OpenAI-compatible vision server (Blackwell cu128 build
      required)
vllm serve Qwen/Qwen3-VL-8B-Instruct \
  --host 0.0.0.0 --port 8090 --max-model-len 32768 --gpu-memory-
      utilization 0.85
```

2. Image generation

2.1 Models and RTX 5090 performance

Model	Precision	RTX 5090 speed	VRAM	Fits 32 GB?	Source
SDXL Base+Refiner	fp16	6.21 s / 1 img (batch1); 15.11 s / 4 imgs (~3.78 s/ img batched)	~24 GB (b1) → ~32 GB (b4)		databasemart SDXL@5090
FLUX.1-dev	fp8 (native Blackwell)	fast; FP8 cuts VRAM ~40% vs BF16	comfortably < 32 GB at fp8		Furkan/SECourses 5090 tests
FLUX.1-schnell	fp8	very fast (few-step distilled)	< 32 GB		ComfyUI built-in
SD3.5-Large	fp8/fp16	benchmarked on 5090 (fp8 vs fp16)	fits at fp8		Furkan/SECourses

Blackwell RTX 5090 has **native FP8 Tensor Cores** → ~40% VRAM reduction and speedup vs BF16 for FLUX/SD3.5. ([databasemart](#); [Furkan Medium](#); accessed 2026-07-06). **2026 successors** (e.g. FLUX.2 / newer SD lineages) are appearing but concrete 5090 VRAM/speed numbers are **UNCONFIRMED** at this date — FLUX.1-dev/schnell remain the safe, well-benchmarked pick.

Best pick for the card: FLUX.1-dev fp8 (quality) + **FLUX.1-schnell** (latency), both in one ComfyUI instance; keep SDXL/SD3.5-Large as alternate checkpoints.

2.2 Serving

ComfyUI is the de-facto engine (graph/workflow API at POST /prompt, results polled or via webhook). Expose an **OpenAI images API** on top with the **comfyui-vllm-omni** node set, which implements POST /v1/images/generations (DALL·E-format) in front of a ComfyUI workflow. Alternatives: SwarmUI (ComfyUI backend), RunPod/Salad serverless templates (FLUX.1-dev-fp8). ([comfyui-vllm-omni](#); [RunPod ComfyUI serverless](#); [Salad ComfyUI deploy](#); accessed 2026-07-06)

3. Video generation (WAN + LTX required)

3.1 Models on 32 GB RTX 5090

Model	RTX 5090 gen time (≈5 s clip)	Resolution	VRAM	Fits 32 GB?	Role
LTX-Video / LTX-2 (LTXVideo 13B / LTX-2.3)	~4 s (near real-time, LTX-2.3); LTX-0.9.5 ~90 s on 4090	up to 4K/50fps (LTX-2.3, official 32 GB target); 768×512 (0.9.5)	12 GB (0.9.5) → 32 GB official (LTX-2.3)	✓	Speed champion , synced audio (LTX-2)
WAN 2.2 14B (fp8)	8–12 min	1280×720 native	24 GB min → full native at 32 GB	✓	Quality leader (photoreal/humans)
HunyuanVideo 1.5 (fp8/Q8)	10–15 min	720×1280	base 60 GB+ FP; Q8 fits 32 GB	✓ (quantized)	natural motion/physics
CogVideoX-5B	—	—	12 GB		older, quality now behind LTX/WAN

Sources: ([Local AI Master — local AI video](#); [AI Magicx — WAN 2.2 vs Hunyuan 1.5 vs LTX 13B](#); [Spheron GPU cloud video AI 2026](#); [Wan-Video/Wan2.2 GitHub](#); [ComfyUI WAN 2.2 native workflow](#); all accessed 2026-07-06). **All four run on a 32 GB RTX 5090** per the Local AI Master compatibility table; note generation time is architecture-bound, not VRAM-bound (a 4090 vs 5090 differ mainly in bandwidth).

Recommendation: run **LTX-Video/LTX-2** as the default fast path and **WAN 2.2 14B fp8** as the quality path — this satisfies the explicit WAN + LTX requirement. Hunyuan 1.5 optional for motion-heavy shots.

3.2 Serving / API exposure

- **WAN 2.2:** ComfyUI native workflow + ComfyUI-WanVideoWrapper (Kijai) nodes; drive via ComfyUI /prompt. Public weights: Wan-AI/Wan2.2-I2V-A14B / Wan2.2-T2V-A14B (HF/ModelScope). ([ComfyUI WAN 2.2](#); [Wan2.2 GitHub](#); [Spheron image-to-video](#); accessed 2026-07-06)
- **LTX-Video / LTX-2:** has an **OpenAI-compatible self-host server pattern** — deploy the open-source model behind a server and point an OpenAI client's `base_url` at it to generate video (Lightricks documents this; community Modal deployment exposes a `...serve.modal.run base_url` usable from the OpenAI SDK). Also available as ComfyUI nodes. ([LTX API & self-hosting help center](#); [HF Lightricks/LTX-2 self-hosted API discussion](#); [LTX docs](#); accessed 2026-07-06)

Because OpenAI has no standardized *video* endpoint, expose video as either (a) the LTX self-host OpenAI-shaped route, or (b) a thin custom `POST /v1/videos/generations async-job` wrapper over ComfyUI. HelixCode should treat video as an async job (submit → poll → fetch artifact).

4. Vectorization / illustration / raster → SVG

Tool	Type	Strength	Serve as
vtracer (visioncortex)	Rust, rule-based, colored high-res raster → SVG	fast, deterministic, no GPU, handles color/photos	CLI microservice (<code>vtracer -- input x.png -- output x.svg</code>) behind FastAPI <code>POST / vectorize</code>
potrace	monochrome bitmap → SVG	sharpest results on B/W line art	same CLI- wrapper pattern
StarVector-8B / -1B (CVPR 2025)	VLM, image/text → SVG code	icons/logos/ diagrams; leverages full SVG syntax (paths, polygons, text)	it is a vision- language model → serve via vLLM/ transformers, / <code>v1/chat/ completions</code> (image in → SVG code out)

Sources: ([visioncortex/vtracer](https://visioncortex.com/vtracer); [StarVector project](#) + [CVPR2025 paper](#); [starvector-8b-im2svg](#); [ComfyUI-Wiki StarVector](#); accessed 2026-07-06)

Recommendation: **vtracer** as the default deterministic vectorizer (CPU, trivially containerized, no VRAM), **StarVector-8B** as the “smart” vectorizer for clean logo/icon/diagram SVG code (fits 32 GB as an 8B VLM), **potrace** for pure monochrome line art. Line-art / sketch and illustration *generation* is best served by the \$2 image models (FLUX/SDXL with line-art LoRAs) rather than a dedicated model.

5. Standard OpenAI-compatible API surface (so HelixCode auto-detects)

Capability	Endpoint	Backend
Chat + vision	POST /v1/chat/completions (image content parts)	vLLM or llama.cpp llama-server (Qwen3-VL)
Model list	GET /v1/models	same
Image generation	POST /v1/images/generations	comfyui-vllm-omni bridge → ComfyUI (FLUX)
Vectorization (StarVector)	POST /v1/chat/completions	vLLM (StarVector-8B)
Vectorization (vtracer/potrace)	POST /vectorize (custom)	FastAPI CLI wrapper
Video generation	LTX self-host OpenAI base_url, or custom async POST /v1/videos/generations	LTX server / ComfyUI (WAN)

Put a lightweight **gateway/router** (e.g. LiteLLM or a small reverse proxy) in front that presents ONE /v1 surface and fans out to the per-capability containers; register the OpenAI base_url + a dummy key with Claude Code / HelixCode so vision + image-gen models are auto-recognized. ([AI Server usage / OpenAI-compatible surfaces](#); [comfyui-vllm-omni](#); accessed 2026-07-06)

6. Rootless Podman deployment

6.1 One-time host setup (CDI GPU passthrough)

```
# 1) Install NVIDIA Container Toolkit (Ubuntu/Debian)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
    sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-
        toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/
    deb/nvidia-container-toolkit.list | \
    sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-
        container-toolkit-keyring.gpg] https://#g' | \
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt-get update && sudo apt-get install -y nvidia-container-
    toolkit

# 2) Generate a CDI spec in USER space (rootless)
mkdir -p ~/.config/cdi
nvidia-ctk cdi generate --output=$HOME/.config/cdi/nvidia.yaml

# 3) Smoke-test rootless GPU access
podman --cdi-spec-dir=$HOME/.config/cdi run --rm \
    --device nvidia.com/gpu=all --security-opt=label=disable \
    nvidia/cuda:12.8.0-base-ubuntu24.04 nvidia-smi -L
```

Sources: ([OneUptime — NVIDIA GPU containers with Podman](#); [Podman Desktop GPU access](#); [NVIDIA CDI support docs](#); accessed 2026-07-06). **Blackwell note:** use a **CUDA 12.8+ base image** and a recent toolkit; older 12.3 base images predate sm_120. If the CDI spec ever goes stale after a driver update, regenerate it (step 2). Rootless caveat: `--security-opt=label=disable` (or a matching SELinux policy) is required; no-cgroups handling is managed by the CDI path.

6.2 Per-capability containers

- **Base images:** vllm/vllm-openai (Blackwell build / cu128 nightly tag — verify sm_120 support before pinning; **UNCONFIRMED** which published tag is Blackwell-ready, build from source with `TORCH_CUDA_ARCH_LIST=12.0+PTX` if needed), ghcr.io/ggml-org/llama.cpp:server-cuda (llama.cpp), a ComfyUI image (e.g. community comfyui + custom nodes), a slim python:3.12 + vtracer/StarVector image.
- **Run pattern (each container):** `podman run -d --device nvidia.com/gpu=all --security-opt=label=disable -v hf-cache:/models -p 8090:8090 <image> <serve-cmd>.`

- **VRAM sharing:** the 5090's 32 GB is time-shared across containers. Cap each engine (`--gpu-memory-utilization` for vLLM, `-nvl/ctx` for llama.cpp) and **stagger heavy jobs** (a WAN 2.2 render + a FLUX batch will not co-reside comfortably). A router that serializes GPU-heavy request classes is the safe design.

6.3 Model-weight volume strategy + §11.4.77 re-obtain mechanism

- Mount **one shared HF cache volume** (`-v hf-cache:/models, HF_HOME=/models`) across all containers so weights download once and are reused.
 - Weights are **large and gitignored** (§11.4.30) — provide the mandated §11.4.77 regeneration mechanism: a non-interactive `scripts/fetch_weights.sh` that runs `huggingface-cli download <repo>` for each model into the cache, plus a `.gitignore-meta/<slug>.yaml` declaring repo id, expected disk size, integrity, `requires-network: true`, `requires-credentials` (HF token for gated repos). Wire it into `setup.sh` post-clone and gate a pre-build check on a `.regenerated/<slug>.ok` stamp. Example weights to declare: Qwen/Qwen3-VL-8B-Instruct(+GGUF), black-forest-labs/FLUX.1-dev, Wan-AI/Wan2.2-I2V-A14B, Lightricks/LTX-Video (LTX-2), starvector/starvector-8b-im2svg.
-

7. Top risks

1. **Blackwell toolchain immaturity (highest).** vLLM / PyTorch require **cu128 + torch ≥ 2.9** for `sm_120`; stock wheels fail with “`sm_120 not compatible.`” Native FP8 works on bare-metal Linux but **not** through WSL2 (`dxgkrnl` fallback). Mitigation: pin cu128 images, prefer bare-metal Linux, keep llama.cpp as a lower-risk fallback engine. ([vLLM #13306](#); [vLLM forum cu128](#))
 2. **Single-GPU VRAM contention.** 32 GB cannot simultaneously hold a 30B VLM + FLUX + WAN 2.2. Concurrent heavy jobs will OOM. Mitigation: a router that serializes GPU-heavy request classes; per-engine memory caps; on-demand model load/unload.
 3. **Non-standard video API + benchmark drift.** No OpenAI videos standard → custom async contract needed, and several perf/VRAM figures come from third-party blogs (2025–2026) that may not match your exact driver/quant. Mitigation: treat cited times as order-of-magnitude, re-benchmark on the actual 5090 before publishing SLAs; wrap video as async jobs.
-

Sources verified 2026-07-06

- Qwen3-VL: <https://github.com/QwenLM/Qwen3-VL> ; <https://codersera.com/blog/qwen3-vl-4b-vs-qwen3-vl-8b-benchmarks-vram-guide/> ; <https://>

- unsloth.ai/docs/models/tutorials/qwen3-how-to-run-and-fine-tune/qwen3-vl-how-to-run-and-fine-tune ; <https://localaimaster.com/blog/qwen-3-vl-local-setup> ; <https://insiderllm.com/guides/vision-models-locally/>
- Qwen2.5-VL / serving: <https://apidog.com/blog/qwen2-5-vl-32b-locally-mlx/> ; <https://lmsysorg.mintlify.app/cookbook/autoregressive/Qwen/Qwen2.5-VL> ; <https://github.com/vllm-project/vllm/issues/24728> ; <https://discuss.vllm.ai/t/speeding-up-vllm-inference-for-qwen2-5-vl/615>
 - llama.cpp multimodal: <https://github.com/ggml-org/llama.cpp/blob/master/docs/multimodal.md> ; <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct-GGUF> ; <https://simonwillison.net/2025/May/10/llama-cpp-vision/>
 - vLLM on Blackwell: <https://github.com/vllm-project/vllm/issues/13306> ; <https://github.com/vllm-project/vllm/issues/37242> ; <https://discuss.vllm.ai/t/vllm-on-rtx5090-working-gpu-setup-with-torch-2-9-0-cu128/1492> ; <https://github.com/vllm-project/vllm/issues/16515>
 - Image gen on RTX 5090: <https://www.databasemart.com/blog/stable-diffusion-benchmark-in-comfyui-on-rtx5090> ; <https://medium.com/@furkangozokara/rtx-5090-tested-against-flux-dev-sd-3-5-ff4e35a07ab8> ; <https://github.com/FurkanGozokara/Stable-Diffusion/wiki/RTX-5090-Tested-Against-FLUX-DEV-SD-35-Large-SD-35-Medium-SDXL-SD-15-AMD-9950X-RTX-3090-TI> ; <https://www.spheron.network/blog/comfyui-gpu-cloud-2026/>
 - ComfyUI API / OpenAI image bridge: <https://github.com/doughtv/comfyui-vllm-omni/> ; <https://www.runpod.io/blog/deploy-comfyui-as-a-serverless-api-endpoint> ; <https://docs.salad.com/container-engine/how-to-guides/ai-machine-learning/deploy-stable-diffusion-comfy> ; <https://docs.servicestack.net/ai-server/usage>
 - Video gen (WAN/LTX/Hunyuan/CogVideoX): <https://localaimaster.com/blog/local-ai-video-generation> ; <https://www.aimagicx.com/blog/open-source-ai-video-models-comparison-2026> ; <https://www.spheron.network/blog/gpu-cloud-video-ai-2026/> ; <https://github.com/Wan-Video/Wan2.2> ; https://docs.comfy.org/tutorials/video/wan/wan2_2 ; <https://www.spheron.network/blog/image-to-video-gpu-cloud-ltx-wan-hunyuan/>
 - LTX self-host / API: <https://support.ltx.studio/hc/en-us/categories/30053379933330-LTX-2-API> ; <https://docs.ltx.video/welcome> ; <https://huggingface.co/Lightricks/LTX-2/discussions/26>
 - Vectorization: <https://github.com/visioncortex/vtracer> ; <https://starvector.github.io/starvector/> ; https://openaccess.thecvf.com/content/CVPR2025/papers/Rodriguez_StarVector_Generating_Scalable_Vector_Graphics_Code_from_Images_and_Te
<https://huggingface.co/starvector/starvector-8b-im2svg> ; <https://comfyui-wiki.com/en/news/2025-03-26-starvector-svg-generation>
 - Rootless Podman + NVIDIA CDI: <https://oneuptime.com/blog/post/2026-03-18-run-nvidia-gpu-containers-podman/view> ; <https://podman-desktop.io/docs/podman/gpu> ; <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/1.14.2/cdi-support.html>